CFGS 2ºDAW - Proyecto Agile

Documentación La Butaca Social-Sprint2

Desarrolladores:

Alberto Llera

Franco Benavides

Rafael Cosquillo

Contenido

1. Introducción....2
1. 2. Objetivos y motivaciones....2
2. Tecnologías y herramientas....3
3. Conceptualización y diagramas....5
3.1 Diagrama entida-relación....5
3.2 Diagrama de casos de uso....7
4. Diseño de interfaces....8
4.1. Página Home....8
4.2. Página películas....13
4.3 Página película....15
4.4 Página búsqueda....17
4.5. Página perfil usuario....19
4.6 Página listas....21
4.7 Página blog....23
4.8 Página about y contacto....25
5. Desarrollo entorno cliente La Butaca Social....27
5.1 Main....28
5.2 Módulo templates....29
5.3 Módulo dedicado a la barra de búsqueda....30
5.4 Modulo carrito y gestión de productos....31
5.5 Modulo dedicado a formularios....33
5.6 Test unitarios y pruebas con selenium....33
5.6.1 Tests formulario login....34
5.6.2 Tests unitarios carrito....36
5.6.3 Tests barra de búsqueda....37
6. Entorno servidor....38
6.1 Conexión con la base de datos....39
6.2 Gestión e integridad de los datos....40
6.3 Procesado de los datos....40
6.4 Sesiones y autenticación....41
7. Conclusión....42

1. Introducción

Aedo Studio es una empresa joven y flexible dedicada al desarrollo de aplicaciones web a medida. En un mercado donde muchas pymes carecen de presencia digital sólida y efectiva, nuestro estudio ofrece soluciones personalizadas, sostenibles y orientadas a las necesidades reales de cada cliente. Nos diferenciamos por un trato cercano, procesos ágiles, optimización de recursos y un compromiso explícito con la sostenibilidad digital, reduciendo huella energética mediante buenas prácticas de desarrollo y hosting verde.

En este contexto nace **Butaca Social**, una aplicación web inspirada en plataformas como Letterboxd, TV Time o IMDb, cuyo objetivo es ofrecer un espacio donde los usuarios puedan descubrir películas, llevar un seguimiento de lo que ven y compartir sus opiniones con otros cinéfilos. Durante la fase de investigación se realizaron entrevistas al cliente y a potenciales usuarios, además de un análisis comparativo de plataformas similares. Esta fase inicial permitió detectar tendencias de diseño contemporáneo, necesidades reales del público objetivo y aspectos esenciales para que la plataforma resulte usable, atractiva y competitiva.

Durante este análisis se identificaron patrones comunes entre las grandes plataformas: uso de diseño oscuro para generar inmersión cinematográfica, organización clara del contenido, predominio visual de portadas y fichas técnicas, así como la integración natural del componente social. Al mismo tiempo, se detectaron puntos débiles en estas plataformas, como publicidad intrusiva, lentitud en ciertos apartados o ausencia de filtros avanzados, que Butaca Social busca mejorar desde su diseño inicial.

El resultado de esta investigación constituye la base del proyecto y establece las directrices de diseño y funcionalidad necesarias para crear una plataforma sólida, accesible y alineada con los valores de Aedo Studio.

1. 2. Objetivos y motivaciones

El proyecto tiene como finalidad principal desarrollar una plataforma web

destinada a cinéfilos, que permita gestionar contenido audiovisual y fomentar la interacción entre usuarios. Para alcanzar este propósito se han definido objetivos claros que combinan aspectos técnicos, experienciales y empresariales.

En primer lugar, se busca crear una aplicación intuitiva y visualmente atractiva, capaz de mostrar películas recientes, fichas completas, listas personalizadas y un sistema de reseñas integrado. La plataforma debe ser rápida, accesible y funcionar correctamente en cualquier dispositivo. Para lograrlo, el diseño se centrará en interfaces limpias, tiempos de carga reducidos y navegación clara, con especial atención a personas con necesidades de accesibilidad.

A nivel empresarial, el proyecto pretende consolidar a Aedo Studio como una compañía capaz de desarrollar productos completos, escalables y sostenibles. La elección de hosting verde, la optimización de imágenes, la reducción de consumo energético y la coherencia estética con el mundo del cine permiten reforzar la identidad ecosocial de la empresa, aportando un valor diferenciador frente a competidores que siguen modelos más estándar de desarrollo.

Finalmente, uno de los objetivos más importantes es integrar correctamente la dimensión social. Butaca Social no debe ser solo un catálogo, sino un espacio donde los usuarios puedan comunicarse, descubrir tendencias, filtrar contenido avanzado y recorrer la plataforma como si formaran parte de una comunidad cinematográfica activa.

2. Tecnologías y herramientas.

En este segundo apartado desglosaremos las tecnologías utilizadas para poder dar forma a La Butaca Social.

El desarrollo y la creación de los diferentes diagramas desde el entidad-relación hasta el diagrama de casos de uso se han llevado a cabo con la herramienta Draw.io.

Para llevar a cabo el diseño de interfaces en primera instancia se han creado prototipos con el fin de poder abocetar las diferentes ideas de los desarrolladores en este apartado técnico. En digital se ha utilizado la herramienta conocida como Figma, se trata de un wireframe que permite crear

bocetos de las futuras páginas de una web, incluso añadiendo elementos interactivos.

Para el desarrollo de la aplicación como tal se ha optado por una organización MVC (Modelo-Vista-Controlador). El proyecto se organiza en torno a esta filosofía, así como entorno a los diferentes lenguajes de marcado y de programación que se han empleado para el desarrollo de la app. La organización sigue esta filosofía: por un lado encontramos un directrorio con vistas denominado view, otro con templates para el encabezado, el pie de página y algunos elementos referentes al carrtito. Otro directorio donde se almacenan los archivos referentes a la parte cliente, que a su vez está organizado por funcionalidades o páginas a las que se asocia. Por otro lado, con respecto a los controladores y al backend es posible encontrar otro directorio llamado PHP.

Esto nos lleva a comentar brevemente los diferentes lenguajes con los que se ha desarrollado el proyecto. Por un lado, para la parte del desarrollo de interfaces se ha utilizado tanto HTML como CSS, así como el framwork Bootstrap. Para el lado cliente el proyecto se ha realizado con JavaScript vanilla, centrado en la manipulación del DOM y siguiendo uno de los principios básicos de React, aunque no presente en el proyecto, de creación de estados con el objetivo de tener toda la información lo mejor organizada posible. Para el lado servidor, los controladores están realizados en PHP. La gestión de la base de datos ha sido posible gracias a MySQL y PHPMyAdmin, ya que para poder contar con un servidor donde lanzar la aplicación se ha hecho uso de XAMPP, y por ende de Apache. De esta manera se ha gestionado el almacenamiento de datos.

Las herramientas de desarrollo que se han empleado han sido esencialmente dos. Por un lado, para la parte de cliente se ha utilizado el IDE Webstrom, mientras que para el desarrollo de interfaces y la parte servidor se ha utilizado Visual Studio Code.

Para el manejo de la información de las películas desde el equipo de desarrallo se ha decidido optar por una API conocida como TMBD. Esta ofrece una gran base de datos sobre el mundo del cine, de tal manera que nos ha facilitado el

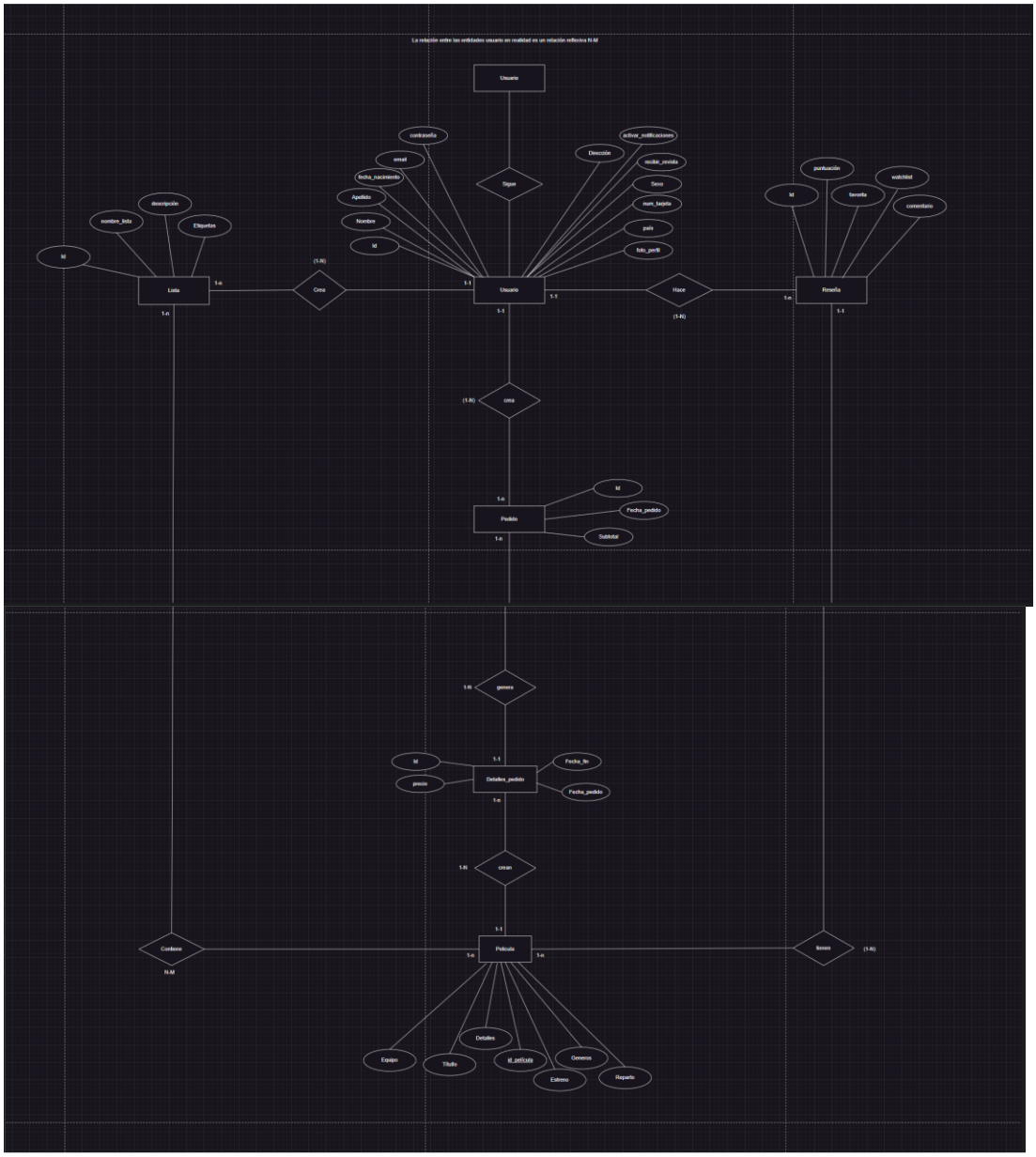
trabajo en este aspecto para poder, de manera dinámica, ofrecer la información sobre las películas al usuario.

Para facilitar el control de versiones entre los diferentes miembros del grupo se ha utilizado un repositorio de GitHub donde se almacena todo el código en la nube. De esta manera es posible para los tres desarrolladores modificar el código existente, así como realizar nuevos añadidos.

Por último, para realizar test en lado cliente se ha utilizado la herramienta conocida como Selenium.

3. Conceptualización y diagramas

3.1 Diagrama entida-relación.



Img 1. Diagrama entidad-relación La Butaca Social.

El diagrama entidad-relación se componer de un total de seis entidades que cubren las funcionalidades de La Butaca Social hasta el momento, esto no significa que con el desarrollo del proyecto la base de datos crezca y sea necesario reformular este diagrama.

La entidad principal de la aplicación es Usuario, ya que sin este ni la app, ni el concepto de la misma tendrían sentido. Cuenta con un total de 13 atributos que cubren la totalidad de la información que debe recabarse de los usuarios para que el flujo de la aplicación sea correcto. Con resto a las relaciones, Usuario tiene relación directa consigo mismo formando una relación reflexiva, ya que los usuarios pueden interactuar entre ellos. El Usuario puede crear listas de películas, aquí se genera una relación con una 1-N, es decir, de uno a muchos. Por otro lado, el Usuario crea también una relación con Reseña, generando una relación con cardinalidad 1-N, pudiendo hacer una reseña como mínimo, muchas como máximo. Por último, la aplicación tiene una funcionalidad para alquilar películas, lo que implica que se puedan realizar pedidos; esto hace que Usuario se relaciones con la entidad Pedido nuevamente con una cardinalidad 1-N.

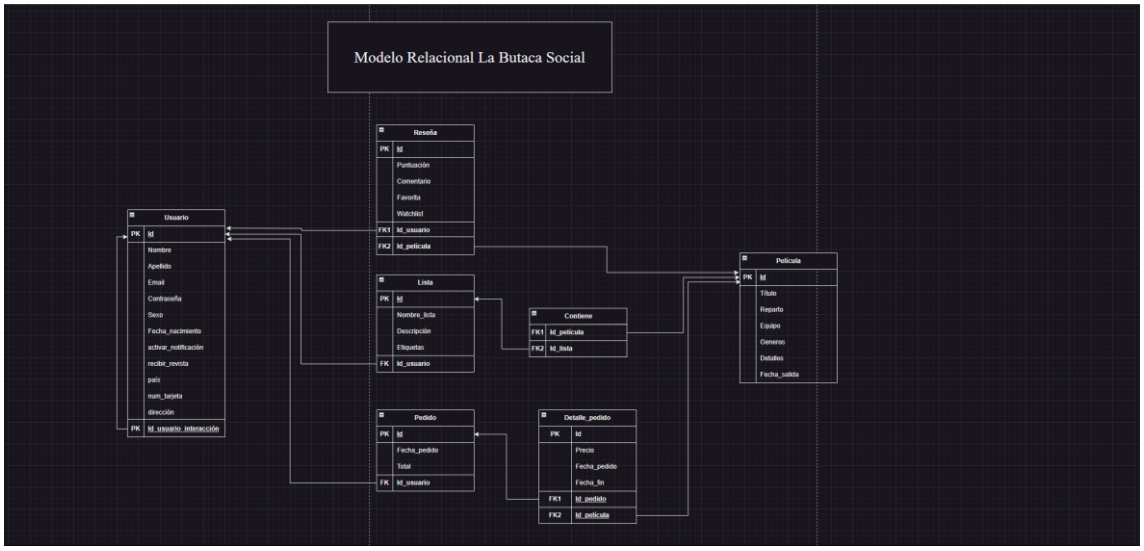
En el diagrama existe también la entidad Lista que cuenta con un total de cuatro atributos. Como ya hemos comentado, se relaciona con Usuario en una relación 1-N. A su vez, establece una relación con Película de carácter N-M, puesto que las listas pueden almacenar muchas películas.

La entidad Reseña tiene relación con Usuario como comentábamos con anterioridad, pero también con Película. Esta tiene cardinalidad 1-N. Además, cuenta con un total de 5 atributos.

Con respecto al flujo de los pedidos encontramos dos entidades fundamentales. Por un lado Pedido que se relaciona estrechamente con Usuario, ya que son estos los que los realizan. Además tiene relación con Detalle_Pedido, esta entidad funciona para crear una tabla intermedia con película que nos ha permitido controlar el flujo de los alquileres y el manejo de los datos para ser mostrados al cliente.

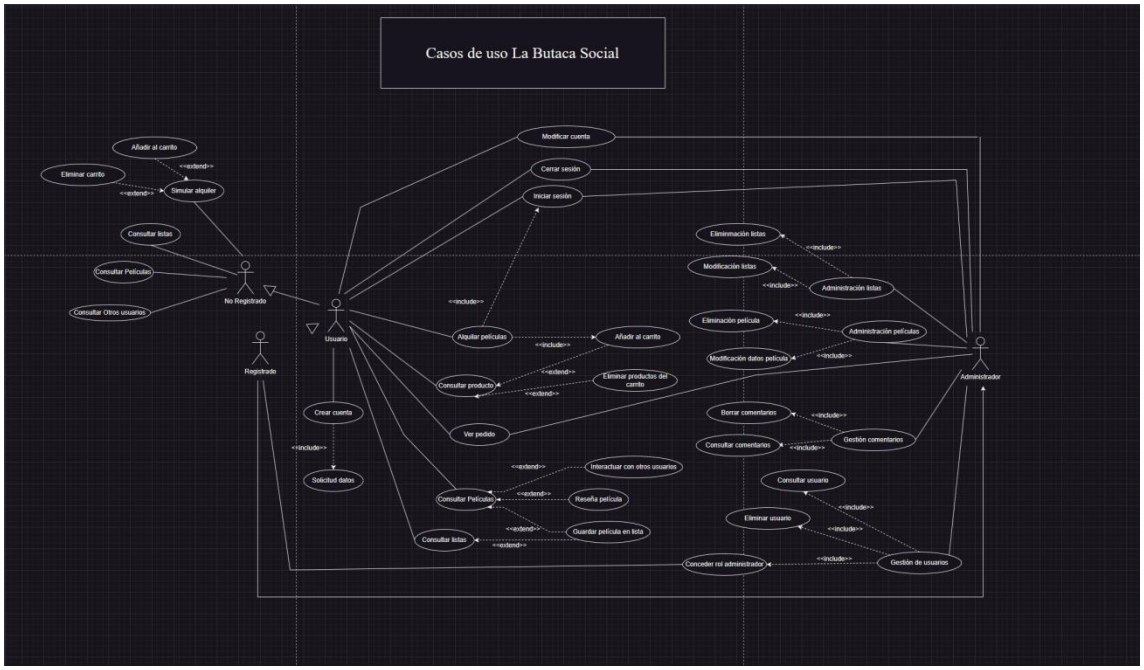
Por último, tenemos la entidad Película que dentro de La Butaca Social es igual de fundamental para la aplicación, ya que es el segundo pilar fundamental. Contiene siete atributos y almacena la información de todas las películas.

Además del diagrama presentado con anterioridad, también se ha desarrollado un diagrama relacional. El objetivo que tiene es dejar aún más claras si caben las relaciones entre las diferentes entidades. Gracias a este tipo de diagrama es más sencillo saber exactamente que entidades comparten una relación y es más sencillo organizar las consultas en el backend de la aplicación.



Img 2. Diagrama relacional La Butaca Social.

3.2 Diagrama de casos de uso.



Img 3. Diagrama de casos de uso de La Butaca Social.

El diagrama de casos de uso define tres actores principales: el usuario no registrado, el usuario registrado y el administrador; así como las funcionalidades a las cada uno tiene acceso.

Primero encontramos el usuario no registrado. Es puede acceder a las películas almacenadas en la base de datos de la Butaca Social, a las listas creadas por los usuarios y consultar usuarios ya registrados en la web. Además, puede simular el alquiler de las películas, que es una de funcionalidades principales de la app. Sin embargo, no podrá completar un alquiler, para eso debe estar alquilado.

Una vez el usuario está registrado y tiene una cuenta en La Butaca Social, su capacidad de interacción con el producto se ve ampliada. Ahora sí que podrá completar el alquiler de películas, cosa que el usuario no registrado no podía. Además, podrá crear listas de películas, realizar reseñas de películas y dar me gusta a las mismas. Por otro lado, tendrá la capacidad de interactuar con otros usuarios como es común en la mayoría de las redes sociales.

Por último, el administrador está por encima del resto de usuarios. Este puede modificar la información de las películas, puede gestionar usuarios, dar permiso de administración a estos si así lo desea, etc. En definitiva tiene control sobre la mayoría de aspectos y puede modificar lugares fundamentales en la web como son los usuarios o las películas.

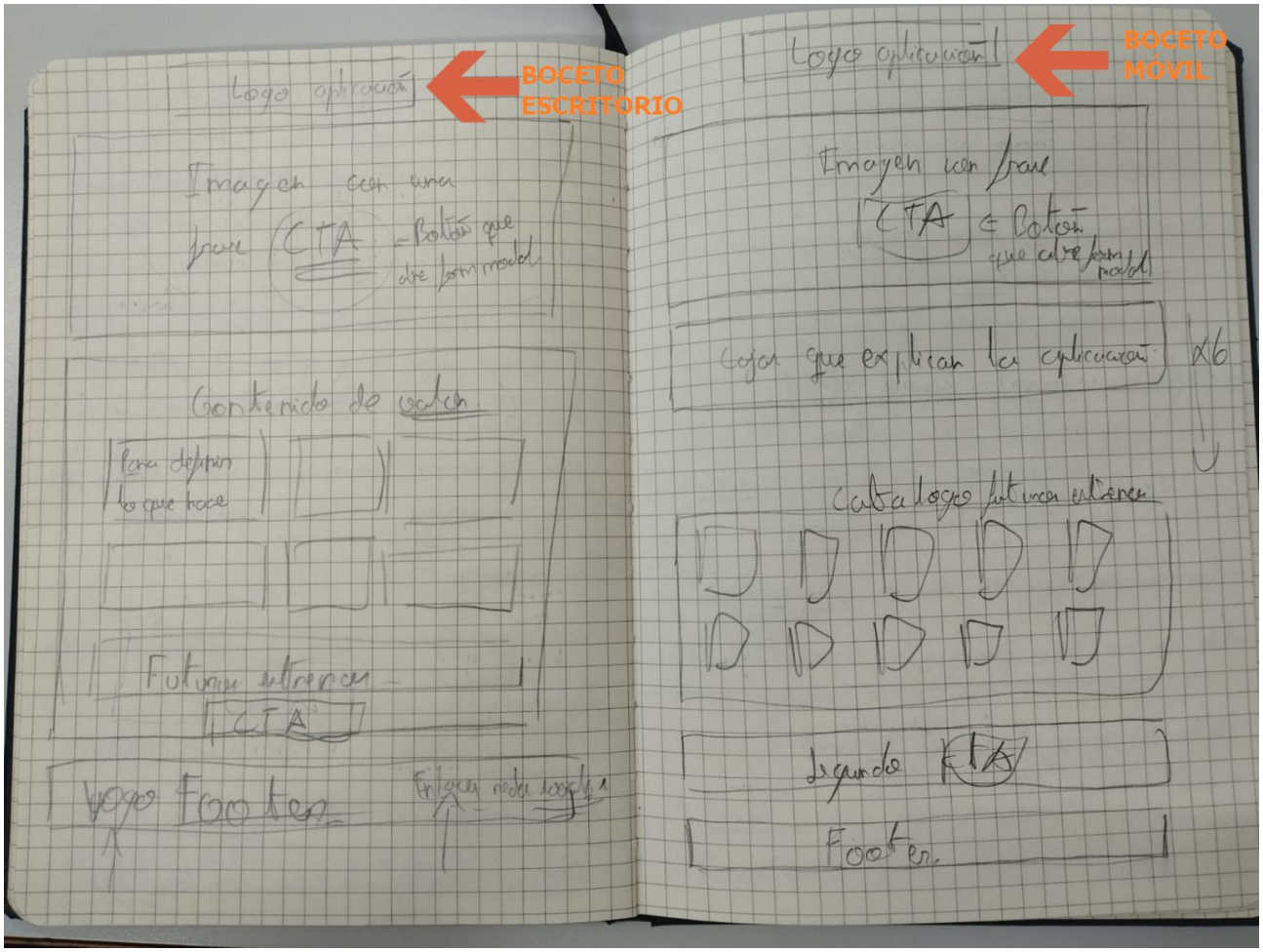
En definitiva, el diagrama muestra de manera visual la manera en la que un usuario puede interactuar con la aplicación. Muestra dependencias y extensiones de los casos de uso, de manera que se entender exactamente el funcionamiento y el flujo de la web.

4. Diseño de interfaces

4.1. Página Home.

Para el desarrollo de la página principal nos hemos basado en un diseño anterior usado para una landing page. A continuación se recoge un análisis detallado del mismo, de las decisiones de diseño, elecciones de color, tipografía, etc.

El boceto (img. 4) nos parece el más adecuado porque cumple bien con los objetivos del cliente: en la parte superior, muestra una imagen principal y un CTA claro, lo que ayuda a captar usuarios desde el primer momento. A continuación se muestra contenido de valor, en este caso hemos decidido incluir un grid con textos informativos que ayuden a entender qué se podrá hacer en nuestra web y una lista de imágenes con posters de las películas más esperadas del próximo año, aunque dependiendo de la estación del año que nos encontremos, esta lista podrá variar para ser más llamativa. Por último, nos encontramos el pie de página, donde vuelve a aparecer nuestra imagen de marca y otro CTA para captar usuarios que hayan sido convencidos a lo largo de la navegación.



Img 4.Boceto página Home.

A nivel visual, hemos utilizado diversos principios de la Gestalt para mantener una coherencia visual y seguir un orden lógico de secciones.

Ley de Proximidad: Los bloques están bien separados: hero image + CTA, explicación, catálogo y pie de página. Esta separación hace que cada sección se entienda como un conjunto independiente, algo clave para que el usuario no se sature. Además, en móvil la proximidad se mantiene en vertical, igual que hace Letterboxd para no romper la lectura y facilitar el continuo seguimiento de los elementos.

Ley de Similitud: Todos los elementos de la lista de películas (los posters) mantienen la misma forma y distribución, lo que ayuda a que el usuario los identifique rápidamente como objetos del mismo tipo. Esta repetición visual refuerza la coherencia.

Ley de Continuidad: El usuario sigue un recorrido bastante natural: empieza en el Hero con CTA, continúa avanzando para obtener más información y termina en otro CTA. El flujo es progresivo y sin saltos raros.

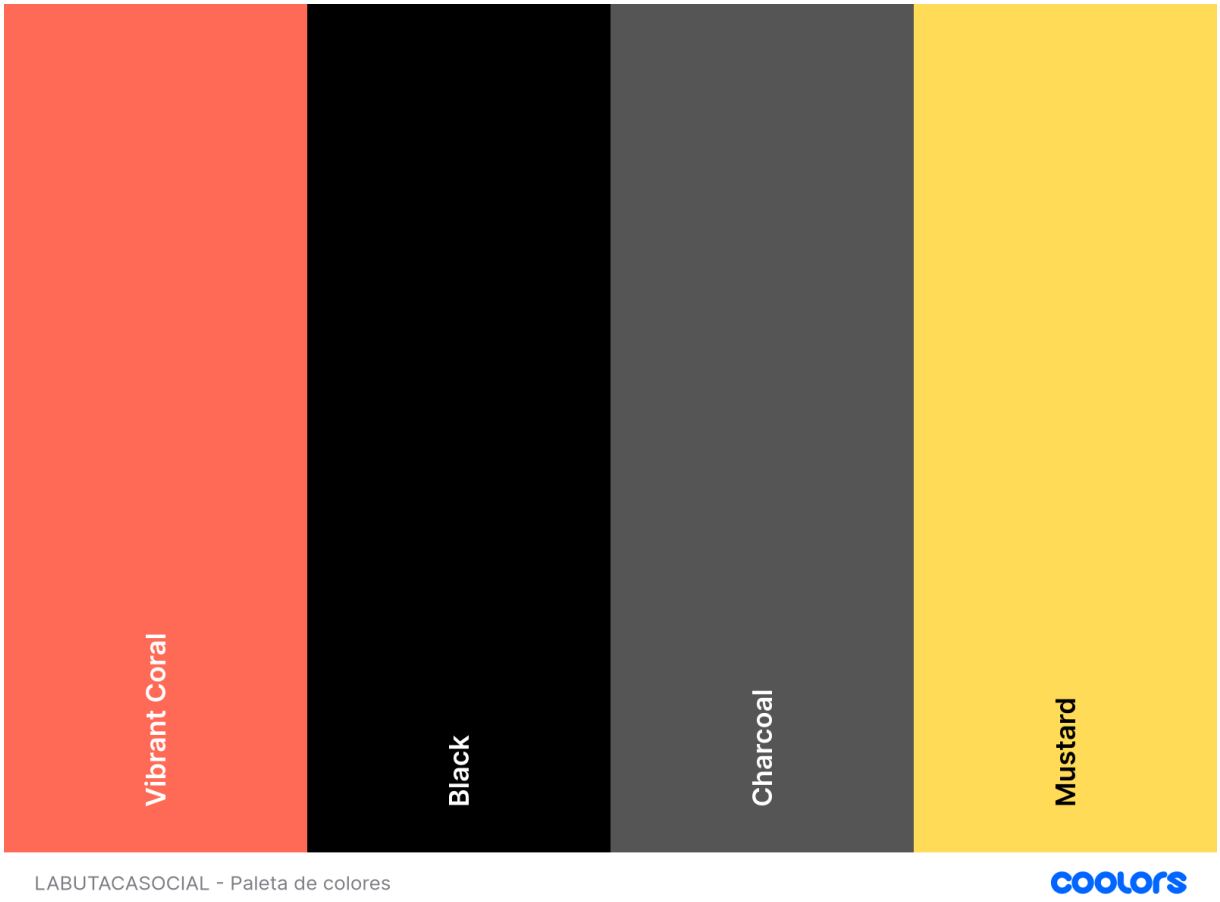
Ley de Figura-Fondo: Los bloques están pensados para destacar el contenido principal (imagen, CTA, catálogo) sobre el fondo, haciendo que la jerarquía visual sea clara. Letterboxd también utiliza esta lógica: bloques limpios donde las carátulas destacan por encima de fondos neutros.

Ley de Pregnancia (buena forma): La composición es simple, simétrica y fácil de procesar ya que no hay ruido visual innecesario. Esto favorece que el mensaje comercial llegue rápido (Regístrate ahora!), igual que Letterboxd, que basa su interfaz en conceptos simples: imagen grande, título, info y acción.

La estructura de bloques está muy clara: cabecera con imagen de marca, imagen-CTA, explicación, catálogo, CTA final y pie de página. Esto facilita mucho que el usuario entienda la jerarquía del contenido sin perderse. En cuanto a la experiencia de usuario, el recorrido es natural: primero entiendes qué es, luego ves cómo funciona y después accedes directamente a la acción. Todo esto también mejora el impacto comercial, porque los CTAs están situados en puntos estratégicos del flujo y el contenido está orientado a convertir de forma sencilla.

La estructura sigue intacta, es muy parecida a la de la landing page. Primero tenemos una cabecera, que es un template y que se compartirá en todas las páginas que conforman la web. En ella podemos encontrar el logo de la marca,

y una barra de navegación con algunas opciones relevantes para la aplicación como puede ser el acceso a las películas, las listas, una barra de búsqueda o el carrito de compra.



Img 5.Paleta de colores La Butaca Social.

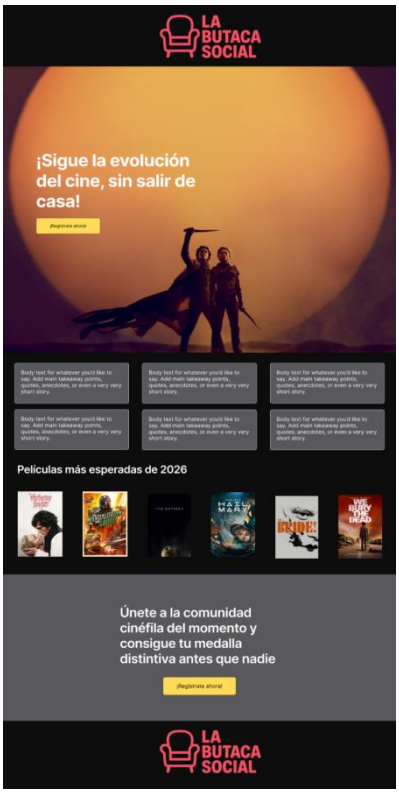
La paleta seleccionada combina **Vibrant Coral (rojo) y Mustard (mostaza) que son colores análogos y sirven para crear armonía en su conjunto. Además, se usa Black (negro) y Charcoal (gris)** para construir una identidad visual que nos permita jerarquizar secciones y contenidos, de forma expresiva junto con los otros dos colores elegidos y ligada al lenguaje cinematográfico del proyecto.

1. **Vibrant Coral (rojo)** se emplea en el logotipo como color principal de marca. Su viveza aporta energía, cercanía y reconocimiento inmediato, reforzando la personalidad propia del proyecto dentro del ámbito audiovisual.

2. **Mustard (mostaza)** actúa como color de acentuación. Su tono cálido y retro destaca elementos interactivos y puntos clave de la interfaz sin resultar estridente, manteniendo una estética cinematográfica.
3. **Charcoal (gris)** se utiliza como fondo secundario y zona de soporte visual. Su neutralidad permite jerarquizar secciones, aportar profundidad y separar contenidos sin competir con los colores principales.
4. **Black (negro)** funciona como base y fondo principal, evocando la experiencia de una sala de cine. Genera contraste, favorece la lectura de elementos visuales y dirige la atención hacia pósters, valoraciones y contenido audiovisual.

En conjunto, esta paleta que hemos escogido ofrece una estética oscura, elegante y contemporánea, alineada con la temática del cine que nos pidió nuestro cliente.

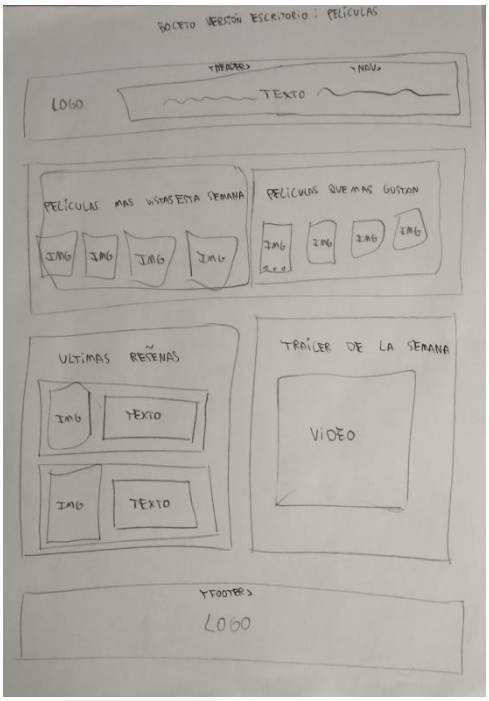
De manera consecuente con los bocetos a mano, también se han realizado bocetos en figma de la página Home, tanto para la versión de escritorio (img. 6) como para la versión de móvil (img. 7).



Derecha. Img 6. Boceto Página Home versión escritorio. Izquierda. Img7. Boceto Página Home versión móvil

4.2. Página películas.

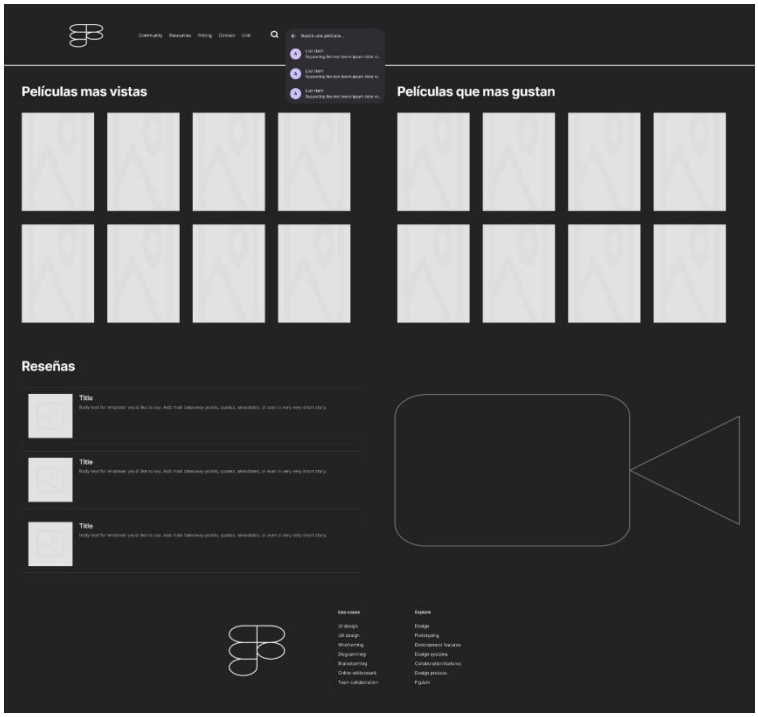
El boceto realizado a mano (img. 8) representa el primer acercamiento al diseño de la interfaz de la página de películas, donde se definen de forma general los distintos bloques que compondrán la vista. En él se puede ver claramente la estructura principal, con un header en la parte superior que incluye el logo y la navegación, seguido de varias secciones diferenciadas como las películas más vistas de la semana, las películas que más gustan, las últimas reseñas y un espacio dedicado al tráiler. En este punto ya se aprecia una correcta aplicación de la ley de proximidad, ya que los elementos que pertenecen a una misma categoría se agrupan dentro de un mismo bloque, facilitando su identificación. Además, también se aplica la ley de región común, puesto que cada sección está delimitada visualmente, lo que ayuda al usuario a entender la separación entre contenidos. Aunque se trata de un diseño simple, es suficiente para definir la jerarquía de la información y sirve como base para el desarrollo posterior.



Img 8.Boceto a mano página Películas.

En la versión digital de escritorio (img. 9) se puede observar cómo este boceto inicial evoluciona hacia un diseño más definido y estructurado. Se mantiene la misma organización de contenidos, pero ahora con una disposición en grid que

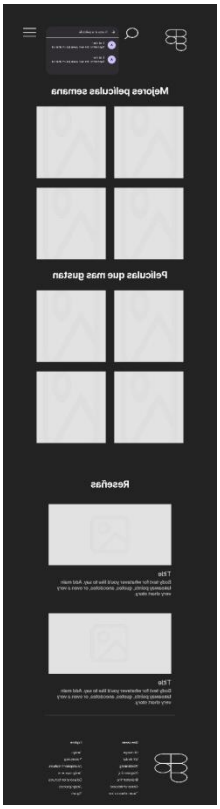
permite mostrar las películas de forma ordenada y homogénea. Aquí se refuerza la ley de similitud, ya que todas las tarjetas de películas comparten el mismo formato, lo que permite al usuario reconocer rápidamente su función. También se sigue aplicando la ley de proximidad, gracias al uso de espacios en blanco que separan claramente unas secciones de otras. El uso de un fondo oscuro junto con elementos más claros genera contraste, lo que mejora la jerarquía visual y facilita la lectura. Además, la inclusión de la barra de búsqueda en el header introduce un elemento interactivo importante que destaca por su posición, guiando al usuario hacia una de las funcionalidades principales de la aplicación.



Img 9.Boceto a mano página Películas.

En cuanto a la versión móvil (img. 10), se adapta el diseño de escritorio a un formato vertical en el que los elementos se reorganizan en una sola columna. Se mantiene la misma jerarquía de contenidos, pero priorizando aquellos elementos más importantes para facilitar su acceso en pantallas más pequeñas. En este caso se aprecia la ley de continuidad, ya que el usuario sigue un flujo natural de navegación mediante scroll. También se mantienen la proximidad y la similitud, lo que permite que el diseño sea coherente respecto a la versión de escritorio. La simplificación del header y la reorganización de los

bloques ayudan a reducir la carga visual, mejorando la experiencia de usuario en dispositivos móviles. En general, el diseño responsive respeta la estructura original y adapta los elementos sin perder coherencia, permitiendo que el usuario entienda la interfaz independientemente del dispositivo que utilice.



Img 10.Boceto versión móvil Películas.

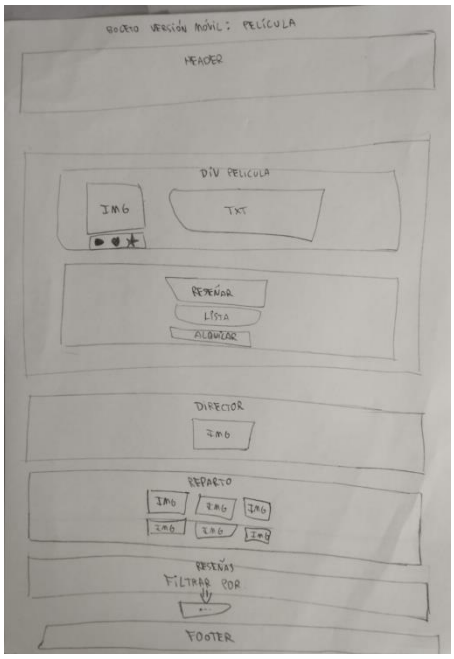
4.3 Página película.

El boceto realizado a mano (img. 11) de la vista de película define la estructura principal de la pantalla, centrada en mostrar la información de una película junto con las acciones que puede realizar el usuario. Se observa una división clara en dos bloques, uno más amplio para la parte visual y descriptiva y otro más pequeño para la interacción, donde aparecen opciones como reseñar, añadir a lista o alquilar.

Esta organización aplica la ley de proximidad al agrupar elementos relacionados. Además, el uso de una estructura clásica con header y footer facilita la comprensión gracias a patrones que el usuario ya conoce.

En la versión de escritorio en formato digital (img. 12) se mantiene esta base, pero con un diseño más trabajado. Se refuerza la jerarquía visual mediante una

imagen principal destacada y un texto descriptivo, mientras que los elementos secundarios como películas relacionadas o reparto se organizan en grid.



Img 11.Boceto a mano película.

Aquí se aplica la ley de similitud, ya que todos estos elementos comparten formato, y la ley de proximidad gracias al espaciado entre bloques. También el contraste entre fondo oscuro y contenido claro ayuda a guiar la atención del usuario hacia las zonas importantes.

Img 12.Boceto versión móvil película.

En la versión móvil (img. 13), el diseño se adapta a una sola columna manteniendo la misma jerarquía. Se prioriza la información principal y las acciones del usuario, facilitando la navegación mediante scroll, lo que refleja la ley de continuidad.

Se mantienen la proximidad y la similitud, consiguiendo una interfaz coherente respecto a escritorio. En general, la adaptación respeta el diseño original y mejora la usabilidad en dispositivos móviles.

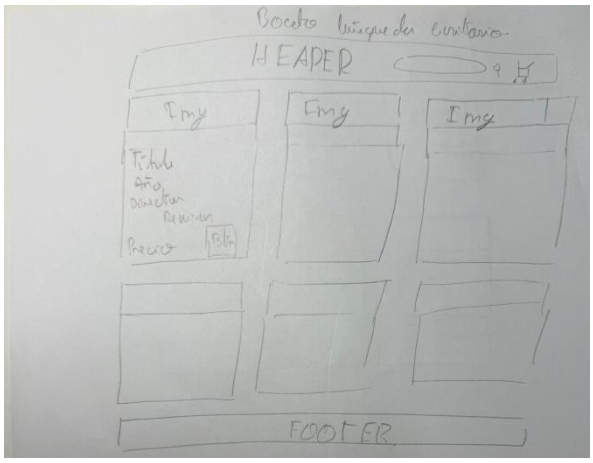
Img 13.Boceto versión escritorio película.

4.4 Página búsqueda

El boceto realizado a mano (img. 14) de la página de búsqueda define una estructura bastante clara basada en un listado de resultados. Se puede ver cómo cada resultado se presenta como una tarjeta que incluye la imagen de la película junto con información básica como título, año, director y precio.

Aquí ya se aplica la ley de proximidad, ya que todos los datos de cada película están agrupados en un mismo bloque, facilitando que el usuario los perciba como una única unidad. Además, la repetición de este mismo patrón en todos

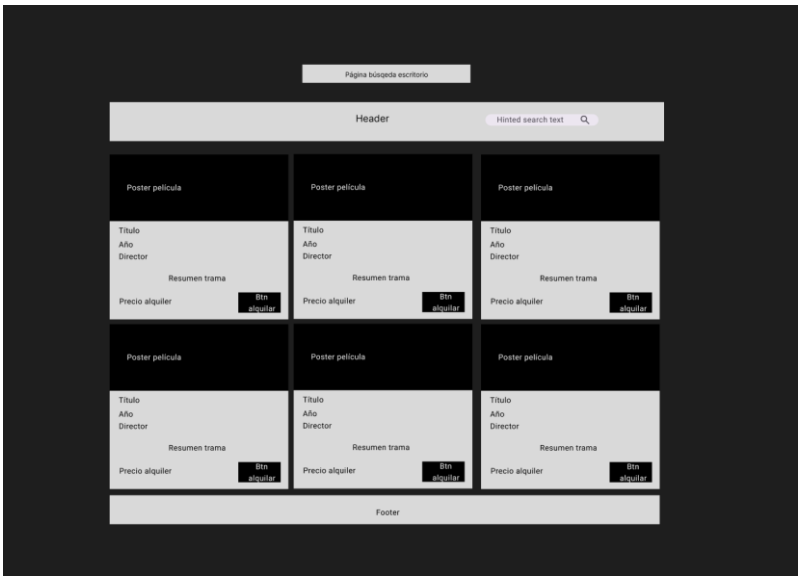
los resultados introduce la ley de similitud, lo que permite reconocer rápidamente que todos los elementos cumplen la misma función dentro de la interfaz. También se mantiene una estructura clásica con header y footer que facilita la navegación.



Img 14.Boceto a mano página de resultados de búsqueda.

En la versión digital de escritorio (im. 15) se refuerza este planteamiento mediante una disposición en grid de varias columnas, lo que permite mostrar varios resultados a la vez de forma ordenada. Cada tarjeta mantiene la misma estructura, lo que potencia la ley de similitud y hace que la interfaz sea predecible para el usuario.

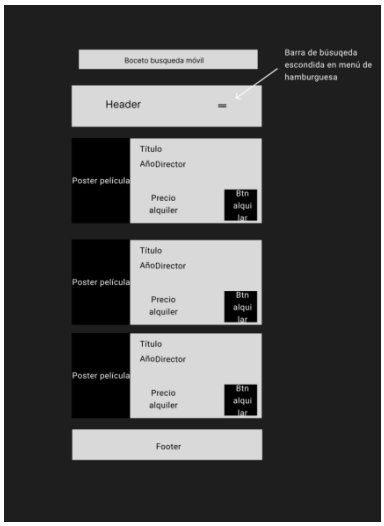
El uso de espacios entre tarjetas aplica la ley de proximidad, separando claramente unos resultados de otros. Además, la barra de búsqueda en el header destaca como elemento principal de interacción, situándose en una posición visible y accesible, lo que ayuda a dirigir la atención del usuario.



Img. 15. Boceto versión escritorio página de resultados de búsqueda.

En la versión móvil (img. 16), el diseño se adapta a una sola columna donde los resultados se muestran en formato vertical. Se mantiene la misma estructura de tarjeta, pero reorganizada para ajustarse al tamaño de pantalla, colocando la imagen y el texto de forma más compacta.

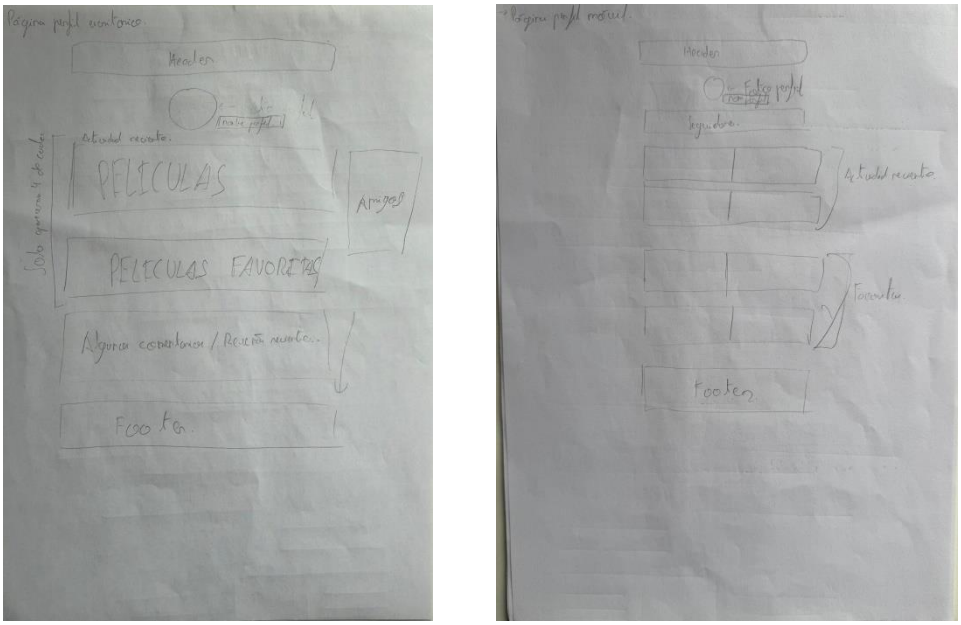
Aquí se aplica la ley de continuidad, ya que el usuario navega de forma natural mediante scroll, y se mantienen tanto la proximidad como la similitud, asegurando coherencia con la versión de escritorio. La decisión de ocultar la barra de búsqueda dentro de un menú también responde a la necesidad de simplificar la interfaz y priorizar el contenido en dispositivos móviles.



Img 16. Boceto versión móvil página de resultados de búsqueda.

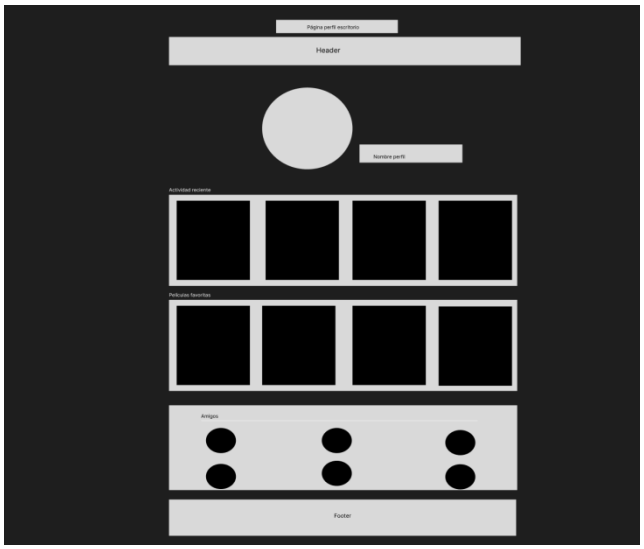
4.5. Página perfil usuario.

El boceto realizado a mano de la página de perfil (img. 17) de usuario plantea una estructura centrada en mostrar tanto la información personal como la actividad dentro de la aplicación. Se distingue una zona superior con el header, seguida del bloque de perfil con la imagen y el nombre, y a continuación varias secciones como actividad reciente, películas favoritas y reseñas. En este punto ya se aplica la ley de proximidad, ya que cada tipo de contenido está agrupado en bloques diferenciados. También se aprecia la ley de región común, al separar visualmente cada sección, lo que facilita que el usuario entienda la estructura general. Además, la presencia de un bloque lateral para amigos introduce una diferenciación clara dentro del layout.



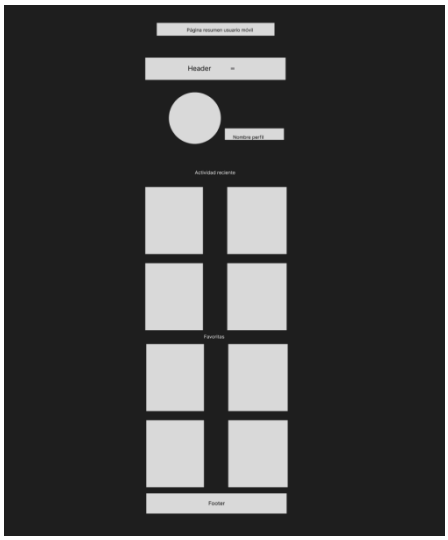
Img 17. Boceto a mano perfil de usuario.

En la versión digital de escritorio (img. 18) se mantiene esta base, pero con una organización más definida. La información del perfil gana protagonismo en la parte superior mediante el tamaño de la imagen, estableciendo una jerarquía visual clara. Las secciones de actividad y favoritas se organizan en grids de tarjetas, aplicando la ley de similitud al repetir el mismo formato. El uso del espacio entre bloques refuerza la ley de proximidad, permitiendo distinguir claramente cada apartado y evitando sobrecargar la interfaz.



Img 18. Versión escritorio perfil de usuario.

En la versión móvil (img. 19), el diseño se reorganiza en una sola columna manteniendo la misma jerarquía. La información del perfil aparece en la parte superior, seguida del resto de secciones dispuestas de forma vertical. Aquí se aplica la ley de continuidad, ya que el usuario navega de manera natural mediante scroll. Se mantienen la proximidad y la similitud, asegurando coherencia con la versión de escritorio. En general, la adaptación simplifica la distribución sin perder claridad, mejorando la experiencia en dispositivos móviles.



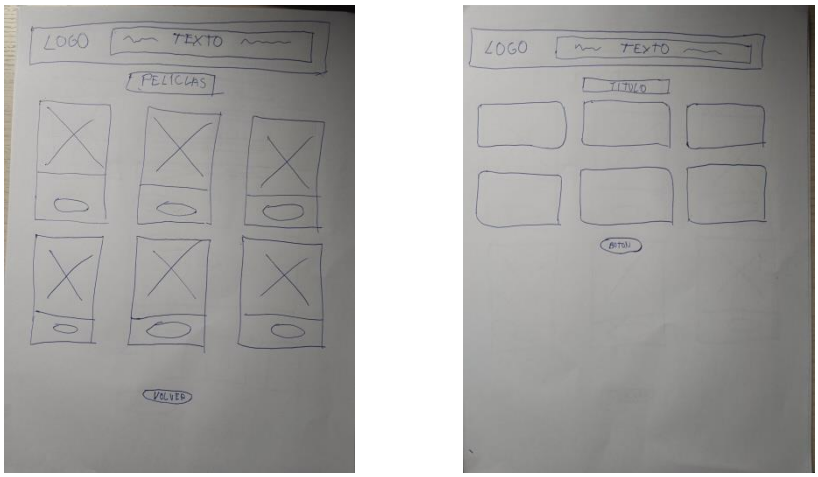
Img 19. Versión móvil perfil de usuario.

4.6 Página listas.

El boceto realizado a mano de la página de listas (img. 20) define una estructura sencilla centrada en mostrar un conjunto de elementos organizados en forma de grid. Seguido del header, elemento que como hemos comprobado aparece en todas las vista, encontramos el título de sección y una serie de tarjetas que representan las películas o listas, con un botón asociado a cada una de ellas.

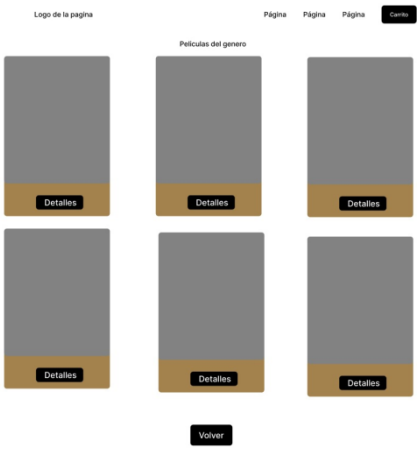
En este diseño ya se aprecia claramente la ley de proximidad, ya que cada tarjeta agrupa su imagen y su botón, haciendo que el usuario lo perciba como un único elemento interactivo. Además, la repetición de este mismo patrón en toda la cuadrícula introduce la ley de similitud, facilitando que el usuario

identifique rápidamente el funcionamiento de cada elemento. La estructura general es simple, pero suficiente para definir la jerarquía básica de la página.



Img 19. Bocetos a mano página Listas.

En el boceto digital de escritorio (img. 20) se mantiene esta idea, pero con un diseño más definido y limpio. Los elementos se organizan en varias columnas, lo que permite aprovechar mejor el espacio y mostrar más contenido de forma ordenada. Aquí se refuerza la ley de similitud, ya que todas las tarjetas tienen el mismo formato, y la ley de proximidad, gracias al espaciado entre ellas. También se puede apreciar una jerarquía clara con el título de la sección en la parte superior y el contenido distribuido debajo, mientras que el botón de volver se sitúa al final como acción secundaria.



Img 20. Versión escritorio página Listas.

En la vista móvil (img. 21), el planteamiento es similar, aunque se simplifica aún más, mostrando únicamente bloques que representan cada categoría. Esto

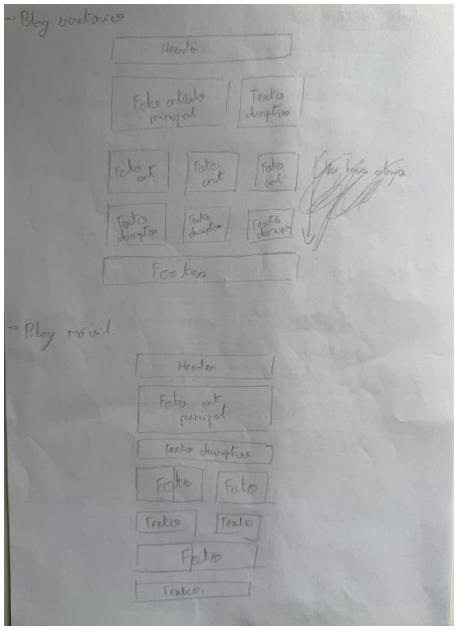
refuerza la claridad visual y reduce la carga de información, manteniendo la coherencia con el resto de la interfaz.

Img 20. Versión móvil página Listas.

En general, el diseño sigue una línea consistente, aplicando principios como proximidad y similitud para organizar el contenido de forma clara. La estructura es fácil de entender y permite al usuario navegar por las listas o géneros de manera intuitiva. Y además es responsive.

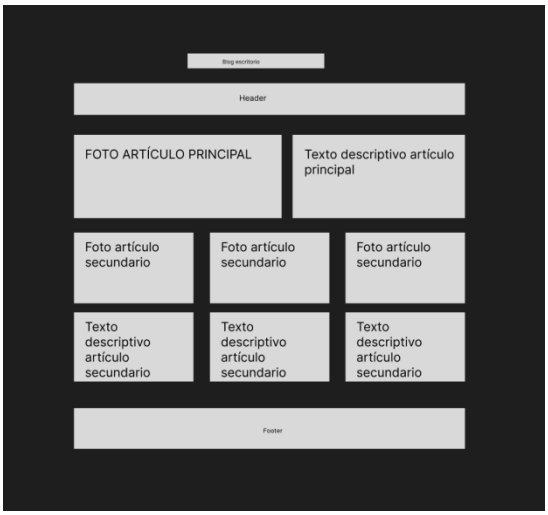
4.7 Página blog.

El boceto realizado a mano de la página de blog (img. 21) define una estructura centrada en destacar un artículo principal seguido de varios artículos secundarios. En la parte superior se sitúa el header, y justo debajo aparece el bloque principal con una imagen grande y su texto descriptivo, lo que indica claramente la importancia de ese contenido frente al resto. Aquí se aprecia la ley de jerarquía visual, ya que el tamaño del elemento principal hace que el usuario lo identifique rápidamente como el contenido más relevante. También se aplica la ley de proximidad en los artículos secundarios, donde cada imagen y su texto están agrupados, permitiendo entender que forman parte del mismo elemento. Además, la repetición de este patrón introduce la ley de similitud, facilitando la lectura, en este caso se busca mostrar artículos recientes para que el usuario siempre tenga información de primera calidad sobre opiniones de cine y crítica cultural del medio.



Img 21. Boceto a mano escritorio y móvil Blog.

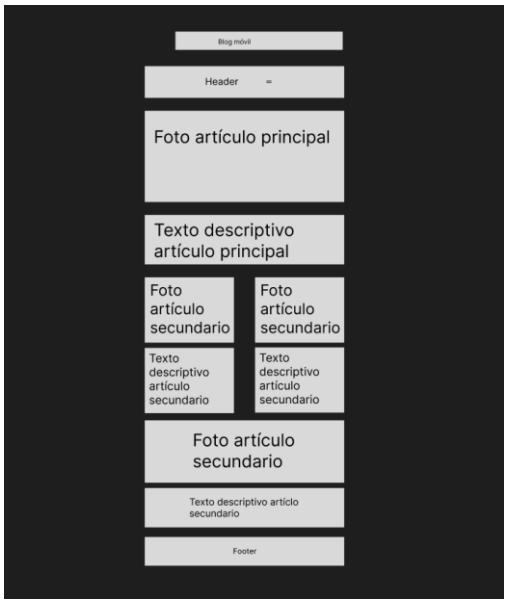
En la versión de escritorio (img. 22) realizada en Figma se mantiene esta idea con una distribución más definida. El artículo principal ocupa una posición destacada en la parte superior, mientras que los secundarios se organizan en una cuadrícula inferior. Se refuerza la ley de similitud al mantener el mismo formato en todos los artículos secundarios, y la ley de proximidad gracias al uso de espacios que separan cada bloque. También se aprecia una estructura clara que guía la lectura de arriba hacia abajo, facilitando la comprensión del contenido.



Img 22.Versión escritorio Blog.

En la versión móvil (img. 23), el diseño se adapta a una disposición vertical donde todos los elementos se colocan en una sola columna. El artículo

principal sigue apareciendo en primer lugar, seguido de los secundarios, respetando la jerarquía establecida. Aquí se aplica la ley de continuidad, ya que el usuario recorre el contenido mediante scroll de forma natural. Se mantienen la proximidad y la similitud, lo que permite conservar la coherencia con la versión de escritorio. En general, la adaptación simplifica la distribución sin perder claridad, mejorando la experiencia en dispositivos móviles.

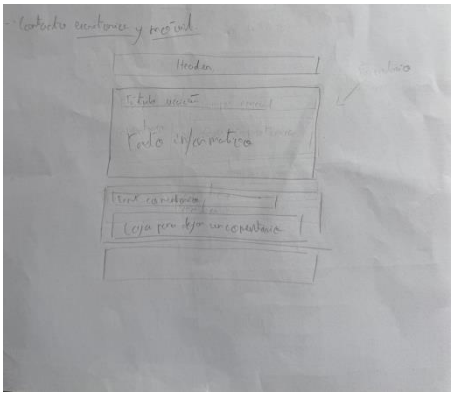


Img 22.Versión móvil Blog.

4.8 Página about y contacto.

El boceto realizado a mano de la página about y contacto (img. 23) plantea una estructura centrada en mostrar información explicativa junto con un formulario de contacto. Se distingue un bloque superior con texto informativo que responde a posibles dudas del usuario (que se reptira varías veces repondiendo todas las dudas), seguido de un bloque inferior donde se sitúa el formulario con un input para el email y una caja de texto para el mensaje.

En este diseño ya se aprecia la ley de proximidad, ya que los elementos del formulario están agrupados entre sí, diferenciándose claramente del contenido informativo. También se aplica la ley de región común, al separar visualmente ambas partes en bloques distintos, lo que ayuda al usuario a entender qué es información y qué es interacción.



Img 23. Boceto a mano págin about y contacto

En la versión realizada en Figma (img. 24) se mantiene esta estructura de forma muy clara, organizando el contenido en dos bloques principales: uno para las secciones informativas tipo preguntas frecuentes y otro para el formulario. La repetición de estas secciones informativas aplica la ley de similitud, ya que todas siguen el mismo formato, facilitando la lectura y comprensión.

El formulario, al estar separado y con un diseño diferenciado, destaca como elemento interactivo, guiando al usuario hacia la acción de contacto. Además, el uso de espacios entre bloques refuerza la ley de proximidad, evitando que la interfaz se vea saturada.

Img 23. Versión Figma de la página about y contacto

Se sigue aplicando la ley de continuidad, ya que el usuario recorre el contenido de arriba hacia abajo de forma natural. En general, se trata de un diseño claro y

directo, donde se prioriza la comprensión del contenido y la facilidad de uso del formulario.

En esta vista de la app, a diferencia de otras páginas, el diseño responsive no cambia la estructura entre escritorio y móvil, manteniendo exactamente la misma disposición. Esto se debe a que la estructura es ya de por sí vertical y sencilla, por lo que no requiere reorganización.

5. Desarrollo entorno cliente La Butaca Social.

El desarrollo de la parte cliente de La Butaca Social como hemos comentado con anterioridad se ha desarrollado con JavaScript vanilla. Para mayor comodidad del control de versiones, el desarrollo y el mantenimiento del código, se ha optado por modularizar esta parte del proyecto. De esta manera se ha separado la lógica en distintos directorios según la funcionalidad que realiza cada parte del código, definiendo la responsabilidad de cada uno de los archivos.

Como punto de entrada principal se encuentra el archivo main.js, que actúa como coordinador general de la aplicación. Desde este fichero se importan los distintos módulos necesarios y se inicializan las funcionalidades cuando el documento termina de cargarse. Aquí se controlan tareas generales como la carga de componentes comunes, la inicialización del buscador, la gestión de formularios, la renderización de películas destacadas o la preparación de la lógica relacionada con el carrito. Por tanto, este archivo no concentra toda la lógica, sino que conecta las diferentes partes de la aplicación en el lado cliente.

A partir de ahí, el código se divide en varios subdirectorios especializados. Por un lado, existe un bloque dedicado al carrito, donde se separa el estado de los productos añadidos, la representación visual del carrito, el resumen de compra y la gestión del pedido final. Por otro lado, hay un directorio orientado a la visualización de películas, encargado de obtener los datos y mostrarlos en pantalla. También aparece un módulo específico para los formularios de registro, donde se agrupan las expresiones regulares, las validaciones, la interacción visual y el control del envío del formulario.

Además, la búsqueda de películas cuenta con su propia carpeta, lo que permite aislar tanto el estado de la búsqueda como la lógica de consulta y la representación de resultados. Esto hace que el sistema de búsqueda sea independiente del resto de funcionalidades y, por tanto, más fácil de modificar o ampliar en el futuro. Junto a ello, también existe un pequeño bloque destinado a la carga dinámica de plantillas comunes, como el encabezado o el pie de página, lo que ayuda a reutilizar estructura HTML en distintas vistas de la aplicación.

En conjunto, esta organización muestra un planteamiento bastante claro del desarrollo en JavaScript, ya que separa la lógica por módulos y evita mezclar responsabilidades distintas dentro del mismo archivo. Gracias a ello, el proyecto mantiene una estructura más profesional y cercana a la forma de trabajo habitual en aplicaciones web reales, donde cada funcionalidad se desarrolla de manera aislada pero coordinada con el resto del sistema.

Realizaremos un comentario detallado de cada una de las partes que componen el JavaScript del proyecto. Comenzaremos por lo más general, es decir, main.js e iremos adentrándonos en cada una de las funcionalidades que han sido programadas para el proyecto.

5.1 Main

El archivo main.js actúa como punto de entrada principal de toda la lógica JavaScript de la aplicación. Su función no es tanto implementar funcionalidades concretas, sino coordinar e inicializar los distintos módulos que componen el sistema. De esta forma, se evita concentrar toda la lógica en un único archivo y se mantiene una estructura más ordenada y escalable.

Desde este fichero se controla el momento en el que la aplicación comienza a ejecutarse, espera a que el DOM esté completamente cargado, para inicializar las diferentes partes de la aplicación, como la carga de componentes comunes, la preparación de eventos globales o la ejecución de funciones necesarias para que cada página funcione correctamente. Esto permite que el resto de módulos se mantengan independientes, ya que no necesitan preocuparse por el momento de arranque.

Además, main.js actúa como punto de conexión entre los distintos directorios del proyecto. Es el encargado de importar los módulos necesarios según la funcionalidad que se quiera activar, como puede ser la gestión del carrito, la visualización de películas, el buscador o los formularios. Gracias a esto, se consigue una separación clara de responsabilidades, donde cada archivo se centra en su tarea, pero todos quedan coordinados desde un único lugar.

En definitiva, main.js cumple un papel fundamental dentro del proyecto, ya que organiza el flujo general de la aplicación y conecta todos los módulos entre sí. Su uso permite mantener una arquitectura más limpia y cercana a una aplicación real, donde existe un punto central que gestiona la inicialización sin sobrecargar el resto del código.

5.2 Módulo templates

El módulo de templates se encarga de gestionar la carga dinámica de elementos comunes de la interfaz, en nuestro caso gestiona tanto el header como el footer, evitando tener que repetir el mismo código HTML en todas las páginas. De esta forma, se consigue una mayor reutilización del código y se facilita el mantenimiento como venimos comentando anteriormente en la documentación de la aplicación, ya que cualquier cambio en estos componentes solo debe hacerse en un único lugar.

Su funcionamiento se basa en la inserción de plantillas HTML dentro del DOM una vez que la página ha sido cargada. Esto permite que todas las vistas compartan la misma estructura base sin necesidad de duplicar contenido, reduciendo posibles errores y manteniendo una mayor coherencia visual en toda la aplicación.

Desde el punto de vista de organización, este módulo está claramente separado del resto porque no gestiona lógica de negocio ni interacción compleja, sino únicamente la carga de estructura visual común. Aun así, tiene un papel importante dentro de la aplicación, ya que asegura que todas las páginas tengan los mismos elementos base y que estos se mantengan sincronizados.

Además, su uso está coordinado desde el archivo principal, que se encarga de llamar a estas funciones cuando es necesario. Esto refuerza la idea de una arquitectura modular, donde cada parte del código tiene una responsabilidad concreta pero todas trabajan de forma conjunta para construir la interfaz completa.

5.3 Módulo dedicado a la barra de búsqueda.

El módulo de la barra de búsqueda se encarga de gestionar una de las funcionalidades principales de la aplicación, permitiendo al usuario encontrar películas de forma rápida a partir de un término introducido que debe coincidir con el título de alguna de las películas almacenadas en la base de datos. Este módulo no solo controla la captura del texto en el input, sino también todo el proceso que va desde la consulta hasta la representación de los resultados en pantalla.

Su funcionamiento comienza detectando la interacción del usuario con la barra de búsqueda. La búsqueda puede dispararse de dos maneras respectivamente ya que existen dos escuchadores listos para detectar la presión de la tecla enter por parte del usuario o el click sobre el icono de la lupa; ambos son típicos en este tipo de componentes de la parte cliente. A partir de ese momento, se lanza una consulta que obtiene una lista de resultados que coinciden con el término introducido. Esta lógica se mantiene separada del resto de módulos, lo que permite que la funcionalidad sea más fácil de mantener y modificar.

Un aspecto importante dentro de este módulo es la gestión del estado de la búsqueda. Se mantiene una variable o estructura que guarda información como el término actual introducido por el usuario y los resultados obtenidos. Esto permite que la aplicación tenga un control claro sobre lo que se está mostrando en cada momento y facilita actualizar la interfaz cuando cambian los datos, evitando inconsistencias o renderizados innecesarios. Además, las diferentes funciones de renderizado que manejan el DOM antes de volver a pintar datos nuevos se encargan de limpiar los anteriores.

En cuanto a la representación de los resultados, se sigue una estructura basada en tarjetas que se repiten, lo que aplica la ley de similitud y facilita que el usuario entienda rápidamente la información mostrada. Además, la organización de estos resultados respeta la ley de proximidad, agrupando los datos de cada película dentro de un mismo bloque, lo que mejora la legibilidad. Por otro lado, para dinamizar la representación de los resultados se ha construido un template en HTML que se clona tantas veces como coincidencias se encuentren en la base de datos con el término introducido por el usuario, lo que facilita la reutilización del código, así como su simplificación porque permite ahorrarnos línea y el mantenimiento del mismo, filosofía que se ha intentado respetar durante la mayor parte del proyecto tanto en el frontend como en el backend.

Otro aspecto importante es que este módulo está desacoplado del resto de funcionalidades, aunque se coordina con el archivo principal para inicializarse cuando es necesario. Esto permite que la búsqueda funcione de manera independiente, sin afectar a otras partes de la aplicación.

En definitiva, la barra de búsqueda no solo mejora la experiencia de usuario al facilitar el acceso al contenido, sino que también está implementada de forma modular, incluyendo la gestión de su propio estado, lo que permite mantener un control claro sobre los datos y su representación en la interfaz.

5.4 Modulo carrito y gestión de productos.

El módulo del carrito se encarga de gestionar todo el proceso relacionado con el alquiler de las películas, acción que como hemos comentado anteriormente está estrechamente relacionada con los usuarios registrados en la web. Su función principal es mantener un control sobre los productos añadidos, permitiendo añadir, eliminar y visualizar los elementos en todo momento. A diferencia de otros módulos más visuales, aquí cobra especial importancia la gestión del estado.

El carrito mantiene una estructura de datos donde se almacenan las películas seleccionadas por el usuario. Este estado es clave, ya que representa en todo momento el contenido actual del carrito y permite que la interfaz se actualice

correctamente cuando se produce algún cambio, como añadir o eliminar un elemento. Gracias a esto, se consigue que la información mostrada sea siempre coherente con las acciones del usuario.

Además de la gestión del estado, este módulo también se encarga de la representación visual del carrito. Para este aspecto se ha decidido incorporar un componente bootstrap conocido como offcanva. Asociado al icono del carrito que está siempre presente en la web porque se asocia al header, el usuario puede navegar por la página pudiendo consultar siempre los productos que va añadiendo, es decir, las películas que desea alquilar. Para el diseño de este componente se ha seguido una agrupación de la información por películas, acompañadas de un botón eliminar que borra el elemento del carrito respetando las leyes de proximidad y las de continuidad que ya hemos desarrollado largo y tendido en anteriores apartados de la parte cliente, pero sobre en el apartado de interfaces (img 24).

Img 24. Detalle de los productos almacenados en el carrito de compra.

Otro de los puntos fundamentales es el cálculo del total de la compra, que se actualiza dinámicamente en función de los productos añadidos. Esto introduce un pequeño flujo dentro de la aplicación, donde el usuario puede revisar su selección antes de finalizar el proceso.

Por tanto, el módulo del carrito combina la gestión de estado con la representación visual, permitiendo al usuario interactuar con los productos de forma clara y manteniendo siempre la coherencia entre los datos y la interfaz.

5.5 Modulo dedicado a formularios.

El módulo registe_form se encarga de gestionar la interacción del usuario con los distintos formularios de la aplicación, principalmente el de registro. Su función es controlar tanto la recogida de datos como su validación antes de ser enviados, asegurando que la información introducida cumple los requisitos establecidos.

Para ello, se utiliza una combinación de validaciones basadas en expresiones regulares y comprobaciones en tiempo real. Esto permite detectar errores mientras el usuario está escribiendo, mostrando mensajes o cambios visuales que indican si los datos son correctos o no. De esta forma, se mejora la experiencia de usuario y se evita el envío de formularios con datos inválidos.

Un aspecto importante de este módulo es la gestión del estado de validación. Cada campo del formulario mantiene información sobre si su contenido es válido, lo que permite tener un control global del formulario completo. Solo cuando todos los campos cumplen las condiciones, se permite el envío, lo que garantiza la coherencia de los datos.

Además, este módulo también gestiona la interacción visual con el usuario, aplicando estilos o clases CSS en función del estado de cada input. Esto refuerza la claridad de la interfaz, ya que el usuario puede identificar rápidamente qué campos necesita corregir. Por otro lado, se ha añadido la función visual de poder ver la contraseña mientras se rellana el formulario, se ha añadido el icono del ojo, que mediante un escuchador de click permite al usuario comprobar exactamente cuál es la contraseña que está introduciendo.

5.6 Test unitarios y pruebas con selenium.

Con respecto a los modulos de JavaScript se han relizado una serie de pruebas unitarias para poder comprobar la funcionalidad de los diferentes componentes que componen esta parte del proyecto. En las próximas líneas se

desarrollará brevemente los tests ejecutados sobre el formulario de login, la barra de búsqueda y el carrito con Selenium.

Requisitos previos para la ejecución de los tests:

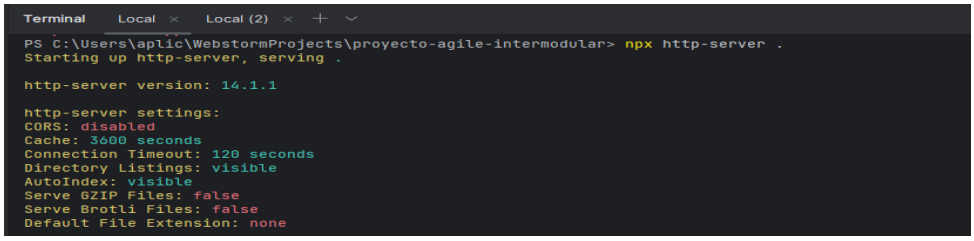
- Node.js (v16 o superior)
- npm o yarn
- Google Chrome instalado (actualizado a la version 143) Abres Google -> menú -> Información de Google.
- Dependencias del proyecto:
    - npm install selenium-webdriver chai mocha --save-dev
- Servidor local para servir los archivos HTML, por ejemplo:
    - npx http-server .

5.6.1 Tests formulario login.

flujo_correcto_test.js - Verifica que el formulario pueda enviarse correctamente con datos válidos.

validaciones.js - Comprueba que se muestren errores cuando los campos tienen datos inválidos (nombre, correo, contraseña, confirmación, fecha).

El primer paso es iniciar el servidor local en el terminal del proyecto de WebStorm (img 25):



Img 25. Iniciación del servidor en la consola de WebStorm.

El siguiente paso consiste en abrir un segundo terminal y usas los siguientes comandos para ejecutar los tests:

- npx mocha tests/validaciones_test.js --timeout 60000 --reporter spec
- npx mocha tests/flujo_correcto_test.js --timeout 60000 --reporter spec

Cada test abrirá una ventana de Chrome, interactuará con el formulario y cerrará el navegador al finalizar. Dejando en el terminal con los mensajes que

se pueden observar en las próximas capturas de pantalla que se adjunta (img 26).

```
PS C:\Users\aplic\WebstormProjects\proyecto-agile-intermodular> npx mocha tests/flujo_correcto_test.js --timeout 60000 --reporter spec


  Formulario correcto de LABUTACASOCIAL

DevTools listening on ws://127.0.0.1:49351/devtools/browser/513cd937-fcca-4181-9641-ee528fd97da3
    √ Debería enviar el formulario correctamente con datos válidos (2568ms)


  1 passing (3s)

PS C:\Users\aplic\WebstormProjects\proyecto-agile-intermodular>
```

```
Terminal   Local ×   Local (2) ×   +   ⌄

PS C:\Users\aplic\WebstormProjects\proyecto-agile-intermodular> npx mocha tests/validaciones_test.js --timeout 60000 --reporter spec


  Verificación de errores - formulario LABUTACASOCIAL

DevTools listening on ws://127.0.0.1:58729/devtools/browser/a63ba473-7e58-4a7c-8405-1a0731ee6e44
[10956:11596:1208/183505.130:ERROR:components\device_event_log\device_event_log_impl.cc:198] [18:35:05.130] USB: usb_service_win.cc:10
ntrado el elemento. (0x490)
    √ Mostrar error si el nombre tiene más de dos palabras o caracteres inválidos (2245ms)
[10956:8756:1208/183505.497:ERROR:google_apis\gcm\engine\registration_request.cc:292] Registration response error message: DEPRECATED_
[10956:8756:1208/183505.534:ERROR:google_apis\gcm\engine\registration_request.cc:292] Registration response error message: PHONE_REGIS
[10956:8756:1208/183505.539:ERROR:google_apis\gcm\engine\registration_request.cc:292] Registration response error message: PHONE_REGIS
    √ Mostrar error si el correo es inválido (2024ms)
    √ Mostrar error si la contraseña no cumple requisitos (2043ms)
    √ Mostrar error si la confirmación de contraseña no coincide (2136ms)
    √ Mostrar error si el correo es inválido (2024ms)
    √ Mostrar error si la contraseña no cumple requisitos (2043ms)
    √ Mostrar error si la confirmación de contraseña no coincide (2136ms)
    √ Mostrar error si la fecha de nacimiento es del futuro (2194ms)


  5 passing (12s)

PS C:\Users\aplic\WebstormProjects\proyecto-agile-intermodular>
```

Img 26. Iniciación del servidor en la consola de WebStorm.

Test de flujo correcto

1. Abrir la página index.html .
2. Hacer click en el botón que abre el modal.
3. Completar el formulario con datos válidos:

    Nombre: Juan Pérez

    Correo: juanperez@gmail.com

    Contraseña: Contrasenna123!

    Confirmar contraseña: Contrasenna123!

    Fecha de nacimiento: 1990-01-01

    Seleccionar radio y checkboxes

    Enviar el formulario y verificar que aparezca mensajeExito .

Tests de validaciones

Ingresar datos inválidos en cada campo y verificar que aparezca el mensaje de error correspondiente (errNombre, errCorreo, etc.).

Cada test abre y cierra el modal para asegurar independencia.

5.6.2 Tests unitarios carrito.

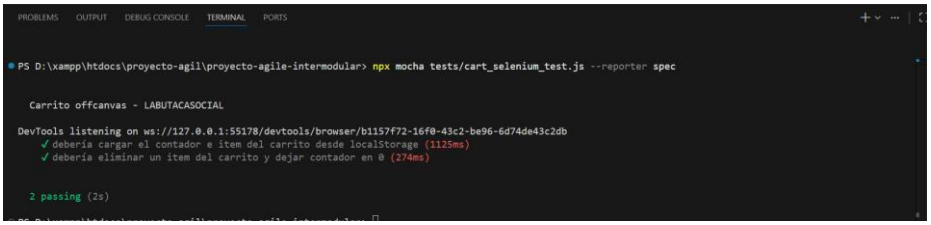
Para las pruebas del carrito se ha preparado un conjunto de tests end-to-end del carrito utilizando Selenium junto con Mocha y Chai. Su objetivo es simular el comportamiento real de un usuario en el navegador y comprobar que el carrito funciona correctamente tanto a nivel visual como a nivel de estado interno.

En el archivo se define un entorno de pruebas donde se lanza un navegador Chrome de forma automática, normalmente en modo headless, lo que permite ejecutar las pruebas sin necesidad de abrir una ventana visible. Además, se configuran variables como la URL base del proyecto y las claves utilizadas en localStorage, lo que permite controlar el estado del carrito desde el propio test.

Uno de los aspectos más importantes del código es que manipula directamente el localStorage para simular que el usuario ya tiene productos en el carrito. De esta forma, no es necesario navegar por toda la aplicación para añadir elementos, sino que se prepara el estado inicial y se comprueba cómo la interfaz reacciona ante él. Esto permite que las pruebas sean más rápidas y más controladas.

En cuanto a las pruebas en sí, se validan dos comportamientos clave. Por un lado, se comprueba que al cargar la página, el contador del carrito y el contenido se actualizan correctamente en función de los datos almacenados. Por otro lado, se verifica que al eliminar un producto, tanto la interfaz como el estado interno se actualizan correctamente, dejando el carrito vacío y eliminando los datos del almacenamiento.

Respecto a los resultados obtenidos por consola, se puede ver que ambas pruebas se ejecutan correctamente, mostrando el mensaje “2 passing”. Esto indica que los dos casos de prueba han sido superados sin errores, lo que confirma que la funcionalidad del carrito se comporta como se espera en estos escenarios (img 27). Para la ejecución de los tests en consola es necesario ejecutar el siguiente comando: **npx mocha tests/cart_selenium_test.js --reporter spec**.



Img. 27. Detalle terminal VS Code con el comando y los resultados del test sobre el carrito

5.6.3 Tests barra de búsqueda.

Al igual que se han realizado pruebas end-to-end del carrito, también se ahn preparado unos tests que comprueban la barra de búsqueda, donde se valida el comportamiento real de esta funcionalidad desde el punto de vista del usuario. Nuevamente utiliza Selenium para automatizar el navegador, permitiendo interactuar con los elementos del DOM como si fuese un usuario real, y Mocha junto con Chai para estructurar las pruebas y verificar los resultados.

Al igual que en el caso anterior se ha definido un código que se lanza en el navegador de Google Chrome, en modo headless para acceder a la interfaz gráfica de forma automática. Además, se incluye una función auxiliar que permite obtener elementos visibles en pantalla, teniendo en cuenta que la aplicación tiene versiones distintas para escritorio y móvil. Esto es importante porque la barra de búsqueda puede estar en diferentes contenedores según el dispositivo, y el test debe adaptarse a ambos casos.

Las pruebas se centran en validar tres comportamientos clave de la barra de búsqueda. En primer lugar, se comprueba que al introducir un término válido y hacer clic en la lupa, la aplicación redirige correctamente a la página de resultados, incluyendo el parámetro de búsqueda en la URL. En segundo lugar, se valida que si el término introducido es demasiado corto, en este caso de menos de dos caracteres, no se produce ninguna navegación, lo que indica que existe una validación previa. Por último, se comprueba que el comportamiento es el mismo al utilizar la tecla enter, asegurando que ambas formas de interacción funcionan correctamente.

En cuanto a los resultados obtenidos por consola (img 28), se puede observar que las tres pruebas se han ejecutado correctamente, mostrando el mensaje “3 passing”. Esto indica que todos los escenarios definidos se cumplen sin errores, confirmando que la barra de búsqueda funciona como se espera. Para ejecutar estos tests es necesario escribir en la terminal ya sea de VS Code o de WebStorm el siguiente comando: **npx mocha tests/search_bar_test.js --reporter spec**.

```
PS D:\xampp\htdocs\proyecto-agil\proyecto-agile-intermodular> npx mocha tests/search_bar_test.js --reporter spec


  Barra de búsqueda - LABUTACASOCIAL

DevTools listening on ws://127.0.0.1:62908/devtools/browser/e526fb37-e144-4f09-95bc-c6d44a1381b7
    ✓ debería navegar a search.php al buscar con la lupa (377ms)
    ✓ no debería navegar si la búsqueda tiene menos de 2 caracteres (1023ms)
    ✓ debería navegar al buscar con Enter (192ms)


  3 passing (2s)
```

Img. 27. Detalle terminal VS Code con el comando y los resultados del test sobre barra de búsqueda

6. Entorno servidor

El entorno servidor de la aplicación está desarrollado en PHP, como ya se ha comentado con anterioridad, y actua como intermediario entre la interfaz de usuario y la base de datos. A diferencia de la parte cliente, donde se centra todo en la interacción y la visualización, en el servidor se procesan las peticiones, se gestionan los datos y se controla el flujo interno de la aplicación.

El backend de La Butaca Social no sigue una arquitectura completamente estructurada como podría ser un modelo MVC estricto, pero aun así se pueden identificar diferentes responsabilidades dentro del código. Existen archivos encargados de la conexión a la base de datos, otros que gestionan operaciones concretas como consultas o inserciones, y scripts que responden a acciones del usuario, como el envío de formularios o la gestión de sesiones.

La comunicación entre cliente y servidor se realiza principalmente a través de peticiones HTTP, donde el frontend solicita información o envía datos, y el servidor responde procesando estas peticiones. Esto permite, por ejemplo, obtener información de usuarios, gestionar datos relacionados con la aplicación o realizar operaciones que requieren persistencia en base de datos.

Además, el servidor también tiene un papel importante en la gestión de la seguridad y la integridad de los datos, ya que es el encargado de validar y procesar la información antes de almacenarla o devolverla al cliente. Esto asegura que la aplicación funcione de forma coherente y que los datos se mantengan consistentes.

En general, aunque la organización del backend no sea completamente estricta, cumple con las funciones necesarias para soportar la aplicación, gestionando la lógica de negocio y permitiendo la comunicación entre el frontend y la base de datos de forma funcional.

6.1 Conexión con la base de datos.

El módulo de conexión a la base de datos es una de las partes fundamentales del backend, ya que es el encargado de establecer la comunicación entre la aplicación y el sistema donde se almacenan los datos. A través de esta conexión, el resto del código puede realizar operaciones como consultas, inserciones, actualizaciones o eliminaciones de información.

En este proyecto, la conexión se realiza mediante PHP utilizando un sistema estándar como es PDO y mysqli, lo que permite interactuar con la base de datos de forma segura y estructurada. Para ello, se definen una serie de parámetros básicos como el host, el nombre de la base de datos, el usuario y la contraseña. Estos datos se utilizan para crear una instancia de conexión que posteriormente será reutilizada en el resto de archivos del backend.

Uno de los aspectos importantes de esta implementación es que la conexión suele estar centralizada en un único archivo, lo que permite evitar duplicar código y facilita el mantenimiento. De esta forma, cualquier cambio en la configuración de la base de datos solo debe realizarse en un único punto, sin afectar al resto del sistema.

El uso de este tipo de conexión permite trabajar con consultas preparadas, lo que mejora la seguridad de la aplicación evitando problemas como la inyección SQL. Esto es especialmente importante en aplicaciones donde se manejan datos introducidos por el usuario.

6.2 Gestión e integridad de los datos.

Los archivos encargados de la gestión de datos son los que contienen las consultas directas a la base de datos y permiten trabajar con la información de la aplicación. A diferencia de una arquitectura más estructurada, aquí no existen modelos como tal, sino scripts PHP que ejecutan consultas SQL cuando se necesita acceder o modificar datos.

En estos archivos se utilizan las conexiones previamente definidas para realizar operaciones concretas, como obtener información de usuarios, comprobar credenciales o insertar nuevos registros. Normalmente, estas operaciones se ejecutan a partir de peticiones que llegan desde el frontend, por ejemplo al enviar un formulario o al cargar una vista que necesita datos.

Se puede ver que la lógica está bastante orientada a casos concretos, es decir, cada script responde a una necesidad específica en lugar de existir una capa completamente reutilizable. Esto hace de la parte servidora de La Butaca Social que tenga un código más concentrado y fácilmente localizable para los desarrolladores. Aun así, se mantiene una separación clara entre la parte que accede a datos y la parte visual.

Además, en estas consultas se hace uso de mecanismos que permiten trabajar con la base de datos de forma más segura, como el uso de parámetros o validaciones previas, evitando así problemas como la inyección SQL. Esto es especialmente importante en operaciones donde intervienen datos introducidos por el usuario.

En conjunto, estos scripts representan la capa de acceso a datos del backend, siendo el punto donde realmente se conecta la lógica del servidor con la base de datos y donde se gestionan las operaciones necesarias para el funcionamiento de la aplicación.

6.3 Procesado de los datos

Los scripts encargados de procesar las acciones del usuario son los que actúan como punto de entrada de las peticiones que llegan desde el frontend.

Estos archivos, como por ejemplo, login.php o logout.php reciben datos enviados desde formularios o desde interacciones del usuario y se encargan de procesarlos antes de realizar cualquier operación en la base de datos.

En estos scripts se recoge la información enviada, mediante métodos POST o GET, y se realizan las comprobaciones necesarias antes de continuar con la ejecución. A partir de ahí, se utilizan las conexiones a base de datos para ejecutar las consultas correspondientes, ya sea para insertar datos, validarlos o modificarlos.

Concentran todo el flujo de una acción concreta. Es decir, en un mismo script se puede ver la recepción de datos, su tratamiento y la respuesta final, ya sea mediante una redirección o devolviendo información al frontend. Esto permite tener una visión clara de lo que ocurre en cada proceso dentro de la aplicación.

Además, estos scripts también pueden encargarse de gestionar aspectos como la sesión del usuario o el control de acceso, dependiendo de la funcionalidad que estén implementando. Esto refuerza su papel como intermediarios entre la interfaz y la lógica interna del sistema.

En definitiva, este bloque representa el punto donde el usuario interactúa realmente con el backend, ya que es donde se procesan sus acciones y se ejecutan las operaciones necesarias para dar respuesta a cada petición dentro de la aplicación.

6.4 Sesiones y autenticación.

La gestión de sesiones y autenticación es la encargada de controlar el acceso de los usuarios a la aplicación y mantener su estado mientras navegan por ella. A través de este sistema, el servidor puede identificar si un usuario ha iniciado sesión y qué permisos tiene dentro de la aplicación.

En La Butaca Social, la autenticación se gestiona mediante scripts que procesan el inicio y cierre de sesión. Cuando un usuario introduce sus credenciales, estas se envían al servidor, donde se validan comparándolas con los datos almacenados en la base de datos. Si los datos son correctos, se inicia

una sesión que permite mantener al usuario autenticado durante su navegación.

Las sesiones permiten almacenar información temporal en el servidor, como el identificador del usuario o sus datos básicos, lo que evita tener que validar las credenciales en cada petición. Esto mejora tanto la seguridad como la experiencia de uso, ya que el usuario no necesita iniciar sesión constantemente.

Además, el sistema también contempla el cierre de sesión, donde se eliminan los datos almacenados y se finaliza la sesión activa, asegurando que el usuario deja de estar autenticado en la aplicación.

En conjunto, este bloque permite controlar el acceso a la aplicación y gestionar la identidad del usuario de forma continua, siendo una parte clave para garantizar la seguridad y el correcto funcionamiento del sistema.

7. Conclusión.

Podemos afirmar, sin lugar a duda que La Butaca Social avanza de manera adecuada en su desarrollo. Este sprint 2 nos ha servido para darle forma a la aplicación pero desde luego que con posibilidades de mejora y cambio.

Por un lado, merece la pena destacar los objetivos cumplidos hasta el momento en el que se escriben estas palabras. Hemos conseguido definir la piedra fundacional de la aplicación, es decir, la base de datos, el modelo de negocio y algunas de sus características principales.

Con respecto al desarrollo de la parte cliente y de interfaces hasta el momento hemos elaborado un producto responsive y adaptado a dos dispositivos, por un lado tenemos una versión de escritorio funciona, así como una móvil de todas las vistas que se han llevado a cabo. Con respecto a JavaScript hemos definido una serie de componentes como pueden ser el formulario de login, la barra de búsqueda y el carrito de los alquileres de películas. Todos ellos funcionales y testeados gracias a Selenium como hemos descrito anteriormente en este documento.

En el lado servidor se ha definido la conexión con el frontend de manera satisfactoria, permitiendo el flujo de datos desde la base hasta la parte visible para los usuarios. Además de esto, se ha definido todo un modelo de consultas que permite que este intercambio entre ambas partes del proyecto funcione correctamente. Además, se ha construido todo un entramado de entidades y relaciones, que permiten la persistencia de los datos y la integridad de los mismos, haciendo de La Butaca Social un producto funcional, aunque aún no completo del todo.

Por otro lado, los desarrolladores de AEDO STUDIO se han planteado una serie de retos para el futuro con respecto a La Butaca Social, de tal manera que la aplicación pueda completarse y modernizarse. El primer paso sería una migración completa de JavaScript vanilla a React, un sistema mucho más moderno que facilitaría la renderización y la muestra de datos a los usuarios, creando un modelo por componentes aún más definido del que se ha hecho hasta ahora. Además, proponemos completar aún más funcionalidades las que queda preparada la aplicación como pueden ser la publicación de reseñas o la interacción entre los usuarios.

Por último, cabría destacar la idea de una migración del desarrollo a docker con el objetivo de abandonar XAMPP para continuar con el desarrollo, así como un despliegue de la web. Con respecto a este último punto, el equipo de desarrollo habrá de reunirse para decir se realizará en tecnologías de Microsoft, es decir, en Azure o por el contrario se optará por AWS.

Con todo esto, podemos concluir que el desarrollo de La Butaca Social está en un punto inmejorable, avanza a buen ritmo. Además, los objetivos inmediatos y futuros son claros, lo que facilita aún más los próximos avances y actualizaciones.